

数据结构与算法大型作业一

Part 1: 解题思路 ——从 BF 算法到贪心算法再到模拟退火

通过查阅资料，我们发现该问题是 3SAT 问题，是一个 np 问题。因此我们放弃了使用 BF 算法的思路，而是使用贪心算法 (greedy algorithm)。该算法逻辑如下。首先，我们给所有 X_i 随机赋值。然后我们随机选择一个 X_i ，试着改变它的 bool 值。如果改变后命题成立的个数大于原来成立的个数，我们就接受这个改动，反之不接受。然后我们重复这个步骤。命题成立数会一直增加，在运气好的情况下，我们会最终让 m 个命题全部成立，这样我们就找到了一组解。

此时有两个问题：1. 这样的算法是否一定能找出一组解 2. 这样的算法效率如何

这里我们先研究第二个问题，再用第二个问题的相关结论去研究第一个问题。

为了估计算法的效率，我们考虑这样一个特殊的 3SAT 问题：让每一个表达式都为 $X_i \cup X_i \cup X_i$ (i 取遍 0 到 n)。这样该 3SAT 问题就有且只有一组解 $X_i=1$ ($i=1,2,\dots,n$)。考虑最坏的情况，我们一开始让所有的 x 都取值为 0。我们考虑在这样的情况下，找出解的次数期望和概率。此时这个问题等价于，我们有 n 个数，每次我们随机取一个，把每一个数都至少取到一次的期望和各种次数完成的概率是多少。

我们这样来计算它的期望 (Markov 过程)。让 $E(n,m)$ 表示总共 n 个数，还剩 m 个数没取，要把剩下 m 个数都至少取一遍的次数期望。所以有：

$$E(n,m) = \frac{n-m}{n}[E(n,m)+1] + \frac{m}{n}[E(n,m-1)+1]$$

化简后得到递推式：

$$E(n,m) = \frac{n}{m} + E(n,m-1)$$

由定义 $E(n,0)=0$ ，故我们得到：

$$\begin{aligned} E(n,n) &= \frac{n}{n} + E(n,n-1) \\ &= \frac{n}{n} + \frac{n}{n-1} + E(n,n-2) \\ &= \dots \\ &= \frac{n}{n} + \frac{n}{n-1} + \dots + \frac{n}{2} + n \\ &\sim n \ln n \end{aligned}$$

也就是说，如果该算法能找到解，时间复杂度的期望仅仅是 $O(n \ln n)$

然后我们来估算如果能找出解，大于 m 次仍然找不出的概率是多少。如果找不出解，那就是至少由一个元素一直没有被选中。我们将其中的元素分为两组，一组有 n-1 个，一组有 1 个 (共 n 种分法)，则执行 m 次一直取不到该 1 个元素的概率为 $(\frac{n-1}{n})^m$ 。故整体来看我们有

$$P(x>m) < n(\frac{n-1}{n})^m$$

Rmk: (这里取小于号是因为各种分法之间有重复的情况, 比如运行 m 次有两个元素都一直没取到就是以那两个元素为代表的情况的公共情况)

进一步放缩+取对数再取指数

$$P(x>m) < n(\frac{n-1}{n})^m = e^{\ln n + m \ln(1-\frac{1}{n})} < e^{\ln n - \frac{m}{n}}$$

所以理论上, 我们只需要取两倍的期望 (即 $m=2n \ln n$), 就可以让该 trivial 3SAT 问题跑不出来的概率小于 $e^{-\ln n} = 1/n$ 。

估算完了该算法的效率, 我们开始研究它的可行性。我们将数据(testData5)导入模型中, 然后让他重复 50 次贪心算法, 每次贪心算法的运行次数上界定为 $2n \ln n$ 。此时我们发现, 50 次能成功的次数非常少, 不超过五次。但是我们发现, 运行完之后得到的答案能让大部分的式子被满足 (每次都能有 395/400 左右的式子被满足)。然后我们试着把运行次数提高, 提高到 $10n \ln n$, $50n \ln n$ 甚至是 $100n \ln n$ 。我们发现成功率有提高但是并没有显著的提高。这说明这个算法确实可以效率很高的让大部分式子被满足, 但是不一定有最大的可能性能找出答案。因为它并不像我们一开始考虑的那个 trivial 的 3SAT 问题, 那个问题只有一个极值, 因而就为最大值。但是一般的 3SAT 问题可能有多个极值。比如在一个点

$P_0 (X_1, X_2, \dots, X_n)$ 的邻域内 (指只改变其中一个 X_i 的值的各种情况下), $f(P_0)$

可能是最大的 (即我们说的, P_0 是极大值点)。但是在整个定义域内, $f(P_0)$ 可能不是最大的 (即可以同时改变多个 X_i 使得 $f(P_0)$ 更大。在这种情况下, 采用贪心算法会导致不论运行多少次结果都不会再变大。而我们又已经证明过, 运行 $2n \ln n$ 次就几乎会把所有的元素都取遍。故对贪心算法来说, 运行 $2n \ln n$ 与 $100n \ln n$ 几乎没有什么差别)。

因此我们需要找一个可行性更高的办法。通过查阅资料我找到了模拟退火算法。

贪心算法失效的原因是它只考虑了局部的最优解。而模拟退火算法会以一定概率接受坏解, 使得自变量跳出局部状态, 从而更整体的考虑问题。它已经在理论上被证明是一种以一定概率收敛于全局最优解的全局优化算法。

于是我们就改动, 但是我们仍保留了贪心算法效率高的优势。我们在前 $2n \ln n$ 次运行使用贪心算法, 而后再开始使用模拟退火算法。通过反复试验我们发现这样是有一定可取之处的。因为如果一开始就使用模拟退火, 则在运行 $2n \ln n$ 次的时候仅仅能满足 350/400 左右个表达式。先使用贪心算法, 可以在前 $2n \ln n$ 次使 395/400 个左右的式子被满足, 这个大概率就是局部最优解。然后我们开始用模拟退火算法找全局解。然后通过调整合适的参数, 再次运行 (每次运行重复 50 次)。发现能找出解的概率大概有 17/20。因此我们认为这个算法还是有很大概率能找出来结果, 如果找不出来, 我们就让程序再执行一次, 直到能找出解 (由模拟退火算法的原理, 这个概率收敛)。

Part 2: 实现难点

在实现中我们主要遇到了一下两个困难:

1. 模拟退火参数的确定
2. 如何真正的获得随机数 (并且保证每次不同)

由于我们估算出了贪心算法的期望 ($n \ln n$) 与概率 $P(x > m) < 1/n$; 并且我们前期就使用贪心算法先快速使大部分式子被满足。所以我们猜想我们最终的贪心+模拟退火混合算法的期望也属于同一个量级。通过反复试验, 我们发现上界改为 $50n \ln n$ 能较好的兼顾成功率和效率。此外, 我们要设定一个接受坏解的概率。由于一个变元在平均意义下与 $3m/n$ 个命题有关, 所以我们就直接令 $T=m/n$, 这样分子就相关的命题状态变化数, 分子就是 $1/3$ 倍的总命题数, 比较合理。

第二个难题是如何获得随机数 (因为我们每次要随机改变一个 x 的状态)。我们的想法是用 `rand()` 函数, 并且引入 `ctime` 来给他当种子, 保证不同。但是我们要生成的数仅仅是从 0 到 n ; 所以我们一开始的想法是让 `rand()` 生成的数模去 n 。但这样有一个问题, 因为 `rand()` 是从 0 到 32767, 所以 `mod n` 之后的得数并不是等概率的 (靠前的数可能得到的概率更大)。所以我们的做法是舍弃 32767 后面的部分。我们找打最大的 m 使得 $mn \leq 32767$ 。如果随机出来的 $x < mn$, 我们就 `mod n` 作为随机的结果, 如果 $x > mn$, 我们就再随机一次, 重复上述步骤。这样在一次过程内, 随机的数就是完全随机的了。

但是在运行过程中我们又发现了问题。如果重复执行 50 次, 我们会发现相邻几次的结果总是一样的。经过思考我发现这是因为我们的随机种子是与时间有关, 但是计算机计算的太快, 在某个时刻可能计算好几次, 而这几次数系统认为他们的时间完全一样, 于是他们用的种子完全一样, 导致结果一致, 因此我们再修改代码, 使得种子不仅与时间有关, 还要与已经执行的次数有关, 这样就能保证每次不一样。(要求重复执行的结果不一样是因为前面说过, 不能保证执行一次就可以出结果, 所以我们有可能要重复执行多次直到出现结果。保证每次结果都不一样可以避免冗余的运行时间)。

Part3: 使用的数据结构

做完了上述步骤之后, 我们的程序已经很完整了。但是还有可以优化的地方。在模拟退火过程中, 我们每次随机都要计算新的状态的总命题成立数 (energy), 计算一次需要把 m 个命题全部都计算一点。但实际上, 我们每次随机仅仅改变了一个变元, 在平均的意义下我们至多能影响了 $3m/n$ 个命题的状态 (10 个左右)。在命题个数很多的情况下, 这甚至差了几十倍。因此我们可以进行优化。我们想知道每次的命题成立数是变多还是变少了, 只需要计算与变元 x 相关的命题在 x 改变前后的成立数变多了还是变少了即可。

因此, 我们的想法是, 一开始导入所有的命题时, 顺便记录每个命题变元与那些命题有关。我们使用的二维的 `VecList` 来实现这一点。

我们先定义 `VecList<VecList<int>*> T> V`。然后让 V 前 n 个位置都插入指向空表的指针。然后我们在导入命题的时候, 遇到了一个数据 x , 就把它所在的行数 y 导入 V 中第 x 个元素指向的表。这样到最后, 如果我们想知道第 i 个命题变元与那些命题有关, 我们就只需要把 V 中第 i 个元素指向的表当中的元素遍历一遍即可。这样我们就让 m 次运算减少到了 $2*3m/n$ 次运算。在 $O(n \ln n)$ 的循环下, 这样的时间复杂的减少还是非常可观的。我们测试过, 用 `data5`, 执行 50 次, 这样大约能节省一半的时间。 n 越大效果越明显。

Bibliography:

<https://baike.baidu.com/item/%E5%B8%83%E5%B0%94%E5%8F%AF%E6%BB%A1%E8%B6%B3%E6%80%A7%E9%97%AE%E9%A2%98/4715567>

<https://baike.sogou.com/v319751.htm>

https://www.bilibili.com/video/BV1mT4y117Xa/?spm_id_from=333.337.search-card.all.click&vd_source=88f14d816a9219a00c407972446cfee0